

Bonnes pratiques R

Migration des programmes SAS vers R

Version 2.0

Objectif - standardiser le codage en R, sécuriser la migration SAS vers R, et faciliter l'appropriation par les équipes métier.

Public - développeurs et analystes, équipe de recette, data management, référents métier.

Périmètre - scripts R en traitement par lots et documents Quarto ou R Markdown.

Dikers Amoko

Data Scientist senior · Industrialisation · Product Owner Data & IA

damoko2004.github.io/portfolio

Bonnes pratiques R

Migration des programmes SAS vers R

Version 2.0

Objectif — standardiser le codage en R, sécuriser la migration SAS vers R, et faciliter l'appropriation par les équipes métier.

Public — développeurs et analystes, équipe de recette, data management, référents métier.

Périmètre — scripts R en traitement par lots et documents Quarto ou R Markdown.

Dikers Amoko

Data Scientist senior · Industrialisation · Product Owner Data & IA

damoko2004.github.io/portfolio

Guide de bonnes pratiques R - Migration SAS vers R v2.0

Guide de bonnes pratiques R

Migration des programmes SAS vers R

Version 2.0 — révision majeure

Objectif. Standardiser le codage en R, sécuriser la migration SAS vers R, et faciliter l'appropriation par les équipes métier.

Public cible. Développeurs et analystes, équipe de recette, data management, référents métier.

Périmètre. Scripts R en traitement par lots et documents Quarto ou R Markdown produisant des jeux de données, indicateurs, contrôles et livrables.

Comment utiliser ce guide

Ce document sert trois moments distincts d'un projet de migration.

Avant de commencer. Les chapitres 1, 2 et 12 fixent le cadre : principes, organisation du dépôt, phasage et critères de décision. Ils se lisent une fois, en équipe, et donnent lieu à des arbitrages écrits.

Pendant la réécriture. Les chapitres 3 à 7 sont une référence de consultation. Le chapitre 6, sur les pièges de sémantique, mérite une lecture attentive avant la première ligne de code : il recense les différences entre SAS et R qui ne provoquent aucune erreur et changent pourtant les résultats.

Au moment de la recette. Les chapitres 8 et 9 décrivent la stratégie de validation et les contrôles. Ils s'appliquent programme par programme.

Les recommandations sont classées en trois niveaux.

Niveau	Signification
Obligatoire	Un écart bloque la recette et doit être justifié par écrit
Recommandé	À suivre sauf contrainte documentée
Libre	Au choix de l'équipe, à condition d'être cohérent sur un même projet

1. Principes directeurs

La migration SAS vers R ne consiste pas à réécrire ligne à ligne. Il s'agit de conserver les mêmes règles métier et les mêmes contrôles, tout en adoptant des standards R qui améliorent la lisibilité, la maintenabilité et l'auditabilité.

1.1 Les six principes à respecter

Lisibilité. Un code compréhensible en cinq minutes par un pair qui ne l'a pas écrit.

Reproductibilité. Les mêmes résultats à partir des mêmes entrées, sur poste comme sur serveur, aujourd'hui comme dans deux ans.

Traçabilité. Entrées, sorties, paramètres, versions et journaux clairement identifiés.

Guide de bonnes pratiques R - Migration SAS vers R - v2.0

Robustesse. Erreurs gérées, contrôles qualité systématiques, valeurs manquantes maîtrisées.

Modularité. Fonctions réutilisables, peu de copier-coller, séparation stricte entre lecture-écriture, logique métier et restitution.

Performance pragmatique. Optimiser quand c’est nécessaire, et seulement à ce moment-là.

1.2 Ce qu’on standardise et ce qu’on laisse libre

Obligatoire	Recommandé	Libre selon contexte
Structure de projet, conventions de nommage, gestion des dépendances, journaux, contrôles qualité, format des sorties	Guide de style, outillage de tests et d’intégration continue, packaging interne, patterns de performance	Choix entre tidyverse et data.table, Quarto ou scripts par lots, outils de restitution en aval

1.3 Le principe qui prime sur tous les autres

En cas de conflit entre deux recommandations de ce guide, celle qui protège l’égalité des résultats l’emporte.

Une migration se juge d’abord sur un critère : les chiffres publiés n’ont pas bougé, ou ils ont bougé pour une raison identifiée, documentée et validée par le métier. Un code élégant qui produit un résultat différent sans qu’on sache pourquoi est un échec.

2. Organisation d’un projet R

Une structure stable facilite la migration, la recette et l’exploitation.

2.1 Arborescence type

```
projet/
  README.md
  renv.lock
  config/
    config.yml          # paramètres d'exécution
  R/
    00_config.R         # chargement et validation de la config
    10_io_import.R      # lecture des entrées
    20_transformations.R # logique métier
    30_controles_qa.R   # contrôles qualité
    40_exports.R        # écriture des livrables
  data/
    raw/                # données sources, lecture seule
    ref/                 # référentiels et tables de correspondance
    work/                # intermédiaires, recréables
    out/                 # livrables
  tests/
    testthat/
  docs/
    dictionnaire_donnees.md
    regles_metier.md
    ecarts_sas_r.md      # registre des divergences
  logs/
  scripts/
    setup.R             # bootstrap de l'environnement
```

Obligatoire. `data/raw/` est en lecture seule. Aucun script n’écrit jamais dedans. C’est ce qui garantit qu’une exécution est rejouable à l’identique.

Obligatoire. `data/work/` est recréable intégralement à partir de `data/raw/`. Sa suppression ne doit rien casser d’autre qu’un temps de recalcul.

2.2 Reproductibilité des dépendances

Utiliser `renv` pour figer les versions des packages, et versionner `renv.lock`.

```
renv::init()      # au démarrage du projet
renv::snapshot()  # après chaque ajout de dépendance
renv::restore()   # sur un nouveau poste ou en intégration continue
```

Documenter la version de R et la plateforme cible. Proscrire les installations manuelles non tracées : tout passe par `scripts/setup.R`.

Pourquoi c’est obligatoire. Sans verrouillage des versions, la reproductibilité tient jusqu’à la prochaine mise à jour d’un package. Un changement de comportement dans `dplyr` ou `readr` peut modifier un résultat sans provoquer la moindre erreur.

2.3 Paramétrage et exécution

Les paramètres d’exécution vont dans un fichier unique au format YAML ou JSON, jamais dispersés dans les scripts.

```
# config/config.yml
periode:
  debut: "2025-01-01"
  fin: "2025-12-31"
chemins:
  sortie: "data/out"
contrôles:
  tolerance_arrondi: 0.005
  bloquer_si_ecart: true
```

Valider la configuration au démarrage et échouer immédiatement si elle est incomplète.

```
charger_config <- function(chemin = "config/config.yml") {
  cfg <- yaml::read_yaml(chemin)
  champs <- c("periode", "chemins", "contrôles")
  manquants <- setdiff(champs, names(cfg))
  if (length(manquants) > 0) {
    stop("Configuration incomplete, champs manquants : ",
         paste(manquants, collapse = ", "), call. = FALSE)
  }
  cfg
}
```

Les scripts doivent être appelables en ligne de commande, sans chemin absolu.

```
Rscript R/10_io_import.R --config config/config.yml
```

Centraliser les chemins avec `here` ou un utilitaire interne. Un chemin absolu dans un script est une bombe à retardement : il fonctionne sur le poste de son auteur et nulle part ailleurs.

2.4 Secrets et identifiants

Obligatoire. Aucun identifiant, mot de passe ou chaîne de connexion en clair dans le code ou dans les fichiers versionnés.

```
# Lecture depuis les variables d'environnement
con <- DBI::dbConnect(
  odbc::odbc(),
  Driver = "SQL Server",
  Server = Sys.getenv("DB_SERVER"),
  Database = Sys.getenv("DB_NAME"),
  UID = Sys.getenv("DB_USER"),
  PWD = Sys.getenv("DB_PWD")
)
```

Les variables se déclarent dans un fichier `.Renviron` local, ajouté au `.gitignore`. Un `.Renviron.example` versionné documente les variables attendues sans leurs valeurs.

Vérifier avant le premier commit que `.gitignore` couvre au minimum `.Renviron`, `data/raw/`, `data/work/`, `logs/` et `.Rhistory`.

3. Conventions de style et lisibilité

3.1 Nommage

Choisir une convention et s'y tenir. `snake_case` est recommandé : `date_traitement`, `montant_ht`.

Préfixer les objets par leur rôle quand cela aide : `df_` pour un data frame, `tbl_` pour une table, `vec_` pour un vecteur, `cfg_` pour une configuration.

Nommer les fonctions avec des verbes : `calculer_score()`, `charger_sources()`, `exporter_livrable()`.

Éviter les noms courts ou ambigus comme `x` ou `tmp`, sauf dans un contexte très local d'une ou deux lignes.

Cas particulier de la migration. Conserver les noms de variables métier issus de SAS, même s'ils ne respectent pas la convention. Renommer `CA_HT_N1` en `chiffre_affaires_ht_annee_precedente` rend le code plus lisible mais casse la correspondance avec la documentation existante, les spécifications et les habitudes des utilisateurs. Si un renommage est décidé, il doit être global, documenté dans le dictionnaire de données, et accompagné d'une table de correspondance.

3.2 Structure d'un script standard

1. En-tête : objet, auteur, date, version, entrées, sorties, dépendances, référence de la spécification.
2. Chargement des packages regroupé, options, graine aléatoire, locale si nécessaire.
3. Lecture des entrées avec contrôle d'existence, de schéma et de types.
4. Transformations, organisées en fonctions.
5. Contrôles qualité : comptages, clés, bornes, doublons, totaux.
6. Exports et résumé de fin : volumétrie, emplacement des sorties, durée.

3.3 Formatage

Indentation de deux espaces, jamais de tabulations. Longueur de ligne cible entre 100 et 120 caractères.

Un objet, une responsabilité. Préférer plusieurs fonctions courtes à une fonction géante.

Commenter le pourquoi, pas le quoi. Les règles métier doivent référencer leur spécification.

```
# Correct : explique une décision
# Exclusion des établissements fermés en cours d'année (règle métier RG-042)
df <- df |> filter(statut != "FERME")

# Inutile : paraphrase le code
# On filtre sur le statut
df <- df |> filter(statut != "FERME")
```

Utiliser un formateur automatique (`styler`) et un analyseur statique (`lintr`) dans la chaîne d'intégration.

3.4 Fixer la graine aléatoire

Tout traitement comportant un tirage aléatoire — échantillonnage, initialisation de modèle, validation croisée — fixe sa graine et la documente.

```
set.seed(20251227) # graine du projet, ne pas modifier sans validation
```

Sans cela, deux exécutions du même code produisent des résultats différents, et toute la stratégie de non-régression s'effondre.

4. Données : import, types, valeurs manquantes, encodages

4.1 Import et export robustes

Pour les fichiers texte, spécifier explicitement l'encodage, le séparateur et les marqueurs de valeur manquante. Ne jamais laisser `readr` deviner.

```
df <- readr::read_delim(
  "data/raw/source.csv",
  delim = ";",
  locale = readr::locale(encoding = "UTF-8", decimal_mark = ","),
  na = c("", "NA", "."),
  col_types = readr::cols(
    identifiant = readr::col_character(),
    date_effet = readr::col_date(format = "%d/%m/%Y"),
    montant = readr::col_double(),
    code_activite = readr::col_character()
  )
)
```

Pour les fichiers SAS natifs, utiliser `haven`, puis contrôler formats, libellés et dates.

```
df <- haven::read_sas("data/raw/table.sas7bdat")
```

Pour les gros volumes, envisager Arrow et Parquet, et limiter les lectures-écritures répétées.

4.2 Déclarer les types plutôt que les deviner

Obligatoire. Aucune colonne ne doit voir son type inféré sans contrôle.

Le risque est concret : un identifiant composé de chiffres avec des zéros initiaux devient numérique, perd ses zéros, et la jointure échoue silencieusement. Un code postal `01234` devient `1234`.

```
# Contrôle de schéma à l'entrée
verifier_schema <- function(df, attendu) {
  observe <- vapply(df, function(x) class(x)[1], character(1))
  commun <- intersect(names(attendu), names(observe))
  ecarts <- commun[observe[commun] != attendu[commun]]
  manquants <- setdiff(names(attendu), names(observe))

  if (length(manquants) > 0) {
    stop("Colonnes absentes : ", paste(manquants, collapse = ", "), call. = FALSE)
  }
  if (length(ecarts) > 0) {
    stop("Types inattendus : ",
         paste0(ecarts, " (attendu ", attendu[ecarts],
               ", recu ", observe[ecarts], ")", collapse = " ; "),
         call. = FALSE)
  }
  invisible(TRUE)
}

schema_source <- c(
  identifiant = "character",
  date_effet = "Date",
  montant = "numeric"
)
verifier_schema(df, schema_source)
```

4.3 Valeurs manquantes

R ne connaît que `NA`. SAS distingue le manquant numérique, noté `.`, du manquant caractère, qui est une chaîne vide. S’y ajoutent les manquants spéciaux SAS, de `.A` à `.Z`, et les codes conventionnels du métier.

Obligatoire. Établir une table de correspondance des codes manquants et la versionner.

```
# docs/codes_manquants.md documente l'origine de chaque code
codes_manquants <- c(-9, -99, 9999)

df <- df |>
  dplyr::mutate(
    dplyr::across(
      dplyr::where(is.numeric),
      ~ dplyr::if_else(.x %in% codes_manquants, NA_real_, .x)
    )
  )
```

Obligatoire. Expliciter le traitement des manquants dans toute agrégation, et le tracer.

```
# Le choix de na.rm est une décision métier, pas une commodité technique
total <- sum(df$montant, na.rm = TRUE)
```

Attention : `na.rm = TRUE` sur une somme donne `0` pour un groupe entièrement manquant, là où le métier attend peut-être `NA`. Vérifier ce cas explicitement.

4.4 Encodages

Déclarer l’encodage à la lecture comme à l’écriture. Les fichiers issus de SAS sur plateforme Windows sont fréquemment en Latin-1 ou Windows-1252, pas en UTF-8.

```
# Diagnostic rapide en cas de caractères accentués corrompus
stringi::stri_enc_detect(readLines("data/raw/source.csv", n = 50, warn = FALSE))
```

Normaliser en UTF-8 dès l’import et n’en sortir qu’au moment de l’export, si un système aval l’exige.

5. Transposition SAS vers R : patterns recommandés

5.1 Équivalences courantes

SAS	R	Points d'attention
DATA step, SET, MERGE, BY	dplyr ou data.table	Contrôler les clés et la volumétrie avant et après jointure
PROC SQL	dplyr, dbplyr ou DBI	Si les données sont en base, pousser les calculs côté serveur
PROC SORT NODUPKEY	distinct(), slice_head() après arrange()	Définir explicitement la règle de déduplication
PROC TRANSPOSE	tidyr::pivot_longer(), pivot_wider()	Vérifier le traitement des doublons de clé
PROC MEANS, PROC SUMMARY	group_by() + summarise()	Attention aux na.rm et aux groupes vides
PROC FREQ	count(), janitor::tabyl()	Les modalités absentes disparaissent en R
PROC UNIVARIATE	summary(), quantile()	Les définitions de quantile différent, voir 6.6
PROC RANK	dplyr::min_rank(), dense_rank()	Gestion des ex aequo et des manquants à préciser
FORMAT, INFORMAT	Dictionnaires et fonctions de recodage	Formaliser les tables de correspondance et les versionner
%MACRO, macro-variables	Fonctions R et fichier de configuration	Éviter l'évaluation dynamique, préférer des fonctions pures
ARRAY et boucle DO	Vectorisation, across(), purrr::map()	Une boucle SAS devient rarement une boucle R
RETAIN	cumsum(), cummax(), purrr::accumulate()	Voir 5.3
FIRST. et LAST.	group_by() + row_number() ou n()	Voir 5.3
LAG()	dplyr::lag(), data.table::shift()	Le LAG SAS dépend du flux d'exécution, pas lag() en R
LIBNAME	Chemins et connecteurs DBI	Standardiser via variables d'environnement
ODS, PROC REPORT	Quarto ou R Markdown avec gt ou flextable	Séparer le calcul du rendu
_N	dplyr::row_number()	_N compte les itérations du DATA step, pas les lignes

5.2 Penser en pipeline, pas en DATA step

Le DATA step SAS traite les observations une par une, avec un état persistant entre les lignes. R travaille sur des vecteurs entiers. Transcrire une logique ligne à ligne en boucle R donne un code lent et illisible.

```
# À éviter : transcription littérale d'un DATA step
for (i in seq_len(nrow(df))) {
  if (df$montant[i] > 1000) {
    df$categorie[i] <- "ELEVE"
  } else {
    df$categorie[i] <- "NORMAL"
  }
}

# Recommandé : vectorisation
df <- df |>
  dplyr::mutate(
    categorie = dplyr::if_else(montant > 1000, "ELEVE", "NORMAL",
                             missing = NA_character_)
  )
```

Noter l'argument `missing` : sans lui, une valeur manquante produirait `NA`, ce qui est probablement le comportement souhaité, mais doit être un choix explicite.

Une transformation égale une étape testable. Préférer plusieurs étapes explicites à une étape magique. Éviter les effets de bord : tout doit dépendre des entrées de la fonction.

5.3 Traitement par groupe : le cas le plus délicat

Les constructions `BY` avec `FIRST.` et `LAST.`, ainsi que `RETAIN`, sont les patterns SAS les plus fréquemment mal traduits. Ils méritent une attention particulière et un test unitaire systématique.

Première et dernière observation d'un groupe.

```
proc sort data = travail; by identifiant date_effet; run;
data resultat;
  set travail;
  by identifiant date_effet;
  if first.identifiant then rang = 1;
  if last.identifiant then derniere = 1;
run;
```

```
resultat <- travail |>
  dplyr::arrange(identifiant, date_effet) |>
  dplyr::group_by(identifiant) |>
  dplyr::mutate(
    premiere = dplyr::row_number() == 1,
    derniere = dplyr::row_number() == dplyr::n()
  ) |>
  dplyr::ungroup()
```

Attention. `FIRST.` en SAS dépend du tri effectué juste avant. En R, `arrange()` doit être explicite et reproduire exactement le même ordre, y compris pour les ex aequo. Si l'ordre des ex aequo n'est pas déterministe, le résultat ne l'est pas non plus : ajouter une clé de départage.

Accumulation avec `RETAIN`.

```
data cumul;
  set travail;
  by identifiant;
  retain total 0;
  if first.identifiant then total = 0;
  total = total + montant;
run;
```

```
cumul <- travail |>
  dplyr::group_by(identifiant) |>
  dplyr::mutate(total = cumsum(dplyr::coalesce(montant, 0))) |>
  dplyr::ungroup()
```

Le `coalesce()` est délibéré : `cumsum()` propage `NA` à toutes les lignes suivantes dès la première valeur manquante, alors que le `RETAIN` SAS aurait ignoré le manquant. La règle attendue doit être tranchée avec le métier et documentée.

Report de la valeur précédente.

```
# LAG SAS et lag() R ne sont pas équivalents.
# Le LAG SAS ne se souvient que des lignes où il a été exécuté.
# En R, lag() opère sur toute la colonne.
travail |>
  dplyr::arrange(identifiant, date_effet) |>
  dplyr::group_by(identifiant) |>
  dplyr::mutate(montant_precedent = dplyr::lag(montant)) |>
  dplyr::ungroup()
```

Si le `LAG` SAS était conditionnel — placé dans un `if` —, il n'existe pas d'équivalent direct. Ce cas demande une réécriture explicite de la logique, validée par un test.

5.4 Jointures

```
# dplyr 1.1 et versions suivantes : déclarer la cardinalité attendue
resultat <- gauche |>
  dplyr::left_join(
    droite,
    by = "identifiant",
    relationship = "many-to-one" # échoue si la relation est violée
  )
```

Déclarer `relationship` transforme un produit cartésien silencieux en erreur explicite. C'est le contrôle le moins coûteux et le plus rentable d'une migration.

Obligatoire. Toute jointure est encadrée par un contrôle de volumétrie.

```
controler_jointure <- function(avant, apres, nom) {
  n_avant <- nrow(avant)
  n_apres <- nrow(apres)
  if (n_apres != n_avant) {
    message(sprintf("[%s] volumetrie : %d -> %d (ecart %d)",
                     nom, n_avant, n_apres, n_apres - n_avant))
  }
  invisible(apres)
}
```

5.5 Déduplication

`PROC SORT NODUPKEY` conserve la première observation de chaque groupe selon le tri courant. La règle implicite doit devenir explicite.

```
# Règle documentée : conserver la ligne la plus récente par identifiant,
# et en cas d'égalité de date, celle dont le montant est le plus élevé
resultat <- travail |>
  dplyr::arrange(identifiant, dplyr::desc(date_effet), dplyr::desc(montant)) |>
  dplyr::distinct(identifiant, .keep_all = TRUE)
```

Ce chapitre est le plus important du guide. Il recense les différences qui ne provoquent **aucune erreur** et changent pourtant les résultats. Ce sont elles qui font échouer les recettes, souvent plusieurs semaines après la bascule.

6.1 Les valeurs manquantes dans les comparaisons

C'est la différence la plus dangereuse, parce qu'elle inverse silencieusement une condition.

En SAS, le manquant numérique est considéré comme inférieur à toute valeur. `if montant < 1000` est donc **vrai** quand `montant` est manquant.

En R, toute comparaison impliquant `NA` renvoie `NA`, et un `NA` dans une condition n'est pas traité comme vrai.

```
# SAS : les manquants sont inclus dans le résultat
# R : les manquants sont exclus
df |> dplyr::filter(montant < 1000)

# Pour reproduire le comportement SAS, l'écrire explicitement
df |> dplyr::filter(is.na(montant) | montant < 1000)
```

Obligatoire. Pour chaque filtre portant sur une variable pouvant être manquante, décider et documenter le comportement attendu. Ce point justifie à lui seul une relecture ligne à ligne des conditions de filtrage.

6.2 L'origine des dates

SAS compte les jours depuis le **1er janvier 1960**. R compte depuis le **1er janvier 1970**. L'écart est de **3 653 jours**.

`haven::read_sas()` convertit correctement les colonnes portant un format de date SAS. Le piège concerne les colonnes stockées en numérique **sans format**, ou les valeurs transitant par un export texte intermédiaire.

```
# Conversion explicite d'un numérique SAS en date R
date_r <- as.Date(valeur_sas, origin = "1960-01-01")

# Datetime SAS : secondes depuis le 1960-01-01
datetime_r <- as.POSIXct(valeur_sas, origin = "1960-01-01", tz = "UTC")
```

Contrôle systématique. Après import, vérifier que les dates minimales et maximales sont plausibles. Une date en 2016 au lieu de 2026 signale un décalage de dix ans.

6.3 Les chaînes de caractères de longueur fixe

SAS stocke les variables caractère sur une longueur fixe et complète avec des espaces à droite. Ses comparaisons ignorent ces espaces. R ne le fait pas.

```
"ABC" == "ABC" # FALSE en R, vrai en SAS
```

Conséquence : une jointure sur une clé caractère issue de SAS échoue partiellement, sans message d'erreur, en ne retrouvant simplement pas les correspondances.

Obligatoire. Nettoyer les clés caractère dès l'import.

```
df <- df |>
  dplyr::mutate(dplyr::across(dplyr::where(is.character), stringr::str_squish))
```

Attention également à la **troncature** : une variable SAS déclarée `LENGTH $10` tronque silencieusement toute valeur plus longue. Si la source a été tronquée côté SAS, la migration ne récupérera pas l'information perdue — mais elle doit la signaler.

6.4 Les arrondis

SAS arrondit à l'écart de zéro : `round(0.5)` donne `1`. R applique la norme IEC 60559, dite arrondi au pair le plus proche : `round(0.5)` donne `0`, et `round(1.5)` donne `2`.

Sur un grand volume, l'écart cumulé devient visible dans les agrégats.

```
# Reproduire l'arrondi SAS
arrondi_sas <- function(x, digits = 0) {
  facteur <- 10^digits
  trunc(abs(x) * facteur + 0.5) / facteur * sign(x)
}

arrondi_sas(0.5) # 1
round(0.5)       # 0
```

Obligatoire. Fixer la règle d'arrondi du projet avant les tests, et l'appliquer uniformément. Le choix — reproduire SAS ou adopter la norme R — est une décision métier, pas technique.

6.5 La comparaison de nombres décimaux

Ne jamais comparer deux nombres décimaux avec `==`.

```
0.1 + 0.2 == 0.3      # FALSE
dplyr::near(0.1 + 0.2, 0.3) # TRUE
all.equal(0.1 + 0.2, 0.3)  # TRUE
```

Dans les contrôles de non-régression, toujours comparer avec une tolérance explicite plutôt qu'en égalité stricte.

6.6 Les quantiles

R propose neuf définitions de quantile ; la valeur par défaut est le type 7. SAS en propose cinq, avec le type 2 par défaut pour `PROC UNIVARIATE`.

```
quantile(x, 0.25, type = 7) # défaut R
quantile(x, 0.25, type = 2) # correspond au défaut SAS
```

Sur de petits effectifs, l'écart est visible. Fixer le type et le documenter.

6.7 L'ordre de tri

SAS place les valeurs manquantes **en premier** dans un tri croissant. R les place **en dernier** : c'est le défaut de `order()` et le comportement systématique de `dplyr::arrange()`, quel que soit le sens du tri.

```
# Reproduire le placement SAS des manquants
df |> dplyr::arrange(!is.na(cle), cle)
```

L'ordre de tri des caractères dépend en outre de la locale. `sort()` sur des accents ne donne pas le même résultat en `C`, en `fr_FR.UTF-8` ou sous SAS.

```
# Tri déterministe, indépendant de la machine
withr::with_locale(c(LC_COLLATE = "C"), sort(codes))
```

Recommandé. Fixer `LC_COLLATE` en début de script pour tout traitement dont l'ordre influence le résultat.

6.8 Les modalités absentes

`PROC FREQ` affiche par défaut toutes les modalités d'un format, y compris celles d'effectif nul. `dplyr::count()` ne renvoie que les modalités présentes.

```
# Conserver les modalités à effectif nul
df |>
  dplyr::mutate(categorie = factor(categorie, levels = modalites_attendues)) |>
  dplyr::count(categorie, .drop = FALSE)
```

Un tableau de restitution auquel il manque une ligne est un écart de recette, même si les chiffres présents sont justes.

6.9 Tableau de synthèse des pièges

Piège	Conséquence si ignoré	Contrôle
Manquants dans les comparaisons	Filtrage inversé	Relire tous les filtres
Origine des dates	Décalage de 3 653 jours	Contrôler min et max après import
Espaces de remplissage	Jointures partielles	<code>str_squish()</code> sur les clés
Arrondi	Écart cumulé sur agrégats	Règle fixée et testée
Égalité de décimaux	Faux écarts en recette	Comparaison avec tolérance
Type de quantile	Écart sur petits effectifs	Type fixé explicitement
Ordre des manquants	Première ou dernière observation fausse	Tri explicite
Modalités absentes	Lignes manquantes en restitution	Facteurs avec niveaux déclarés

7. Modularisation et réutilisation

7.1 Règles

Extraire les règles métier en fonctions paramétrées plutôt que de les copier entre scripts.

Créer un package interne lorsqu'un ensemble de fonctions est partagé entre plusieurs projets.

Respecter une interface simple : entrées explicites, sorties explicites, erreurs explicites.

```
#' Calculer le score de risque
#
#' @param df Table des observations, doit contenir `montant` et `anciennete`.
#' @param seuil Seuil de bascule, par défaut celui de la configuration.
#' @return La table d'entrée augmentée de la colonne `score`.
#' @details Implémente la règle de gestion RG-107, version du 2025-11-03.
calculer_score <- function(df, seuil = 1000) {
  stopifnot(is.data.frame(df),
    all(c("montant", "anciennete") %in% names(df)),
    is.numeric(seuil), length(seuil) == 1)

  dplyr::mutate(df, score = dplyr::if_else(montant > seuil, anciennete * 2, anciennete))
}
```

7.2 Package interne

```
usethis::create_package("chemin/vers/monPackage")
usethis::use_package("dplyr")
usethis::use_roxygen_md()
usethis::use_testthat()
usethis::use_vignette("guide_utilisation")
devtools::document()
devtools::check()
devtools::build()
```

Recommandé. Créer le package une fois que trois projets au moins partagent les mêmes fonctions. Le créer trop tôt ajoute une contrainte de versionnement sans bénéfice.

8. Journaux, erreurs et auditabilité

8.1 Journalisation

Un journal par exécution, horodaté, contenant la version du code, la version de R, les packages, les paramètres, la volumétrie et la durée.

```
journaliser_contexte <- function(cfg) {
  message("--- Contexte d'exécution ---")
  message("Date       : ", format(Sys.time(), "%Y-%m-%d %H:%M:%S"))
  message("R         : ", R.version.string)
  message("Plateforme  : ", R.version$platform)
  message("Commit     : ", system("git rev-parse --short HEAD", intern = TRUE))
  message("Parametres  : ", jsonlite::toJSON(cfg$contrôles, auto_unbox = TRUE))
}
```

Trois niveaux : **INFO** pour les étapes, **WARN** pour les écarts tolérés, **ERROR** pour le bloquant.

Obligatoire. Ne jamais journaliser de secret ni de donnée personnelle.

8.2 Gestion d'erreurs

Utiliser `tryCatch()` pour capturer et enrichir les erreurs aux frontières de lecture-écriture.

Échouer tôt : si une entrée manque ou si un schéma est incorrect, arrêter immédiatement plutôt que de produire un résultat partiel.

Rédiger les messages d'erreur en termes métier : quelle règle a échoué, sur quel périmètre, avec quel impact.

```
stop(sprintf(
  "Contrôle RG-042 en echec : %d établissements sans code d'activite sur la periode %s. Traitement interrompu.",
  n_defaults, cfg$periode$debut
), call. = FALSE)
```

9. Tests, recette et stratégie de validation

9.1 Niveaux de tests

Unitaires. Tester une fonction sur des cas simples et des cas limites : table vide, valeurs manquantes, doublons, valeurs extrêmes.

Intégration. Tester un pipeline complet sur un échantillon figé.

Non-régression. Rejouer les scénarios clés et comparer aux résultats attendus.

```
testthat::test_that("calculer_score gere les montants manquants", {
  entree <- data.frame(montant = c(500, NA, 1500), anciennete = c(1, 2, 3))
  resultat <- calculer_score(entree, seuil = 1000)
  testthat::expect_equal(resultat$score, c(1, NA, 6))
})
```

9.2 Stratégie de comparaison avec SAS

1. Définir un jeu de données de référence figé et ses sorties attendues, produits par la chaîne SAS existante.
2. Comparer le nombre de lignes, les clés, les agrégats, les distributions et les fichiers produits.
3. Documenter les tolérances : arrondis, formats de date, ordres de tri, encodage.
4. Tracer toute divergence : cause, décision de correction ou d’acceptation, validation métier.

Obligatoire. Le jeu de référence est figé et versionné. Comparer à une exécution SAS du jour revient à comparer deux cibles mobiles.

9.3 Outils de comparaison

Plutôt que d’écrire des comparaisons à la main, utiliser les packages dédiés.

```
# Comparaison structurelle, très lisible en sortie
waldo::compare(resultat_r, resultat_sas)

# Comparaison de tables avec clé, rapport détaillé
diffdf::diffdf(base = resultat_sas, compare = resultat_r, keys = "identifiant")

# Rapport de comparaison exportable
rapport <- arsenal::comparedf(resultat_sas, resultat_r, by = "identifiant")
summary(rapport)
```

`diffdf` est particulièrement adapté à la recette : il distingue les lignes absentes, les lignes en trop, et les écarts de valeur colonne par colonne.

9.4 Fiche d’écart

Chaque divergence constatée donne lieu à une fiche, conservée dans `docs/ecarts_sas_r.md`.

Champ	Contenu
Référence	Identifiant unique, par exemple <code>ECART-014</code>
Programme	Nom du traitement concerné
Contrôle	Volumétrie, clé, borne ou agrégat
Valeur SAS	Valeur de référence
Valeur R	Valeur obtenue
Écart	Absolu et relatif
Cause	Diagnostic technique
Décision	Corriger, accepter, ou différer
Validé par	Nom et date, côté métier

Ce registre est l’artefact qui permet d’autoriser la bascule, et d’y revenir six mois plus tard quand quelqu’un demandera pourquoi un chiffre a changé.

9.5 Double exécution

Recommandé. Avant la bascule définitive, faire tourner les deux chaînes en parallèle sur au moins deux cycles de production complets, en comparant automatiquement les sorties.

C’est la seule façon de détecter les écarts qui n’apparaissent que sur certaines configurations de données — fin d’exercice, données exceptionnelles, changement de millésime.

10. Performance et industrialisation

10.1 Règles de base

Éviter les boucles sur de gros volumes ; préférer la vectorisation, `dplyr` ou `data.table`.

Guide de bonnes pratiques R - Migration SAS vers R - v2.0

Limiter les copies : sélectionner les colonnes utiles, filtrer tôt, agréger au plus tôt.

Mesurer avant d’optimiser, avec `profvis` ou `bench`, et documenter les optimisations retenues.

10.2 Choix d’outils selon la volumétrie

Volumétrie	Recommandation	Notes
Moins de 5 millions de lignes	<code>dplyr</code> et <code>readr</code>	Suffisant dans la plupart des cas, garder la lisibilité
5 à 50 millions	<code>data.table</code> ou Arrow et Parquet	Surveiller la mémoire, privilégier Parquet pour les échanges
Plus de 50 millions, ou données en base	<code>dbplyr</code> et SQL, traitements côté base	Pousser le calcul au plus près des données

10.3 Intégration continue

Recommandé. Automatiser à chaque modification : formatage et analyse statique, exécution des tests, construction d’un artefact versionné, déploiement reproductible par conteneur ou environnement figé.

11. Documentation, revue de code et gouvernance

11.1 Documentation minimale

- README.** Contexte, prérequis, mode d’exécution, entrées et sorties, structure du projet.
- Dictionnaire de données.** Colonnes, types, règles, sources, fréquences de mise à jour.
- Registre des règles métier.** Chaque règle identifiée, datée, avec le script qui l’implémente.
- Journal des modifications.** Ce qui a changé, pourquoi, et l’impact sur les résultats.

11.2 Revue de code : liste de contrôle

- Nommage et lisibilité conformes au standard
- Contrôles qualité présents et pertinents
- Traitement des valeurs manquantes explicite
- Aucun chemin absolu, aucun secret en clair
- Types déclarés à l’import, schéma contrôlé
- Jointures encadrées par un contrôle de volumétrie
- Graine aléatoire fixée si nécessaire
- Performance raisonnable, lectures-écritures maîtrisées
- Tests présents, au minimum sur les règles sensibles
- Pièges du chapitre 6 vérifiés pour les variables concernées

11.3 Rôles

Rôle	Responsabilité
Référent technique	Standards, revue de code, arbitrages d’architecture
Développeur	Réécriture, tests unitaires, documentation du code
Référent métier	Validation de l’équivalence, décision sur les écarts
Recette	Exécution du plan de tests, tenue du registre d’écarts
Exploitation	Reprise du run, supervision, gestion des incidents

Obligatoire. La validation d’un écart est prononcée par le référent métier, jamais par l’équipe technique seule.

12. Conduite de la migration

12.1 Phasage

Phase 1 — Audit. Inventaire du patrimoine, cartographie des dépendances entre programmes, identification des entrées et sorties, mesure de la complexité. Livrable : un inventaire priorisé.

Phase 2 — Cadrage. Standards, arborescence, environnement, jeu de référence, critères d’acceptation. Livrable : le présent guide adapté au contexte, et le plan de recette.

Phase 3 — Lot pilote. Migrer un programme représentatif mais non critique, de bout en bout. Objectif : calibrer la charge réelle et éprouver la chaîne de validation avant de s’engager sur le reste.

Phase 4 — Industrialisation. Migration par lots, en suivant l’ordre des dépendances. Chaque lot passe la recette avant le suivant.

Phase 5 — Double exécution. Les deux chaînes tournent en parallèle sur au moins deux cycles complets.

Phase 6 — Bascule et décommissionnement. Arrêt de la chaîne SAS, archivage du code source et d’un jeu de référence, libération des licences.

Recommandé. Ne jamais lancer la phase 4 sans avoir terminé la phase 3. La charge estimée sur plan est systématiquement inférieure à la charge réelle mesurée sur le pilote.

12.2 Matrice de décision par programme

Tout ne mérite pas d’être migré.

Situation	Décision
Programme actif, règles documentées	Migrer
Programme actif, règles perdues	Reconstituer les règles avec le métier avant de migrer
Programme exécuté moins d’une fois par an	Migrer en dernier, ou traiter à la demande
Programme sans utilisateur identifié	Ne pas migrer, archiver après confirmation
Programme redondant avec un autre	Fusionner plutôt que migrer deux fois
Sortie remplacée par un outil de restitution	Ne pas migrer, rediriger vers l’outil

Recommandé. Cet inventaire est le premier livrable de la migration. Il révèle souvent qu’une part notable du patrimoine n’a plus d’usage, ce qui réduit d’autant la charge.

12.3 Typologie de complexité

Niveau	Caractéristiques	Effort indicatif
Simple	Lecture, filtre, agrégation, export	Quelques heures
Moyen	Jointures multiples, recodages, formats	Un à trois jours
Complexe	RETAIN, FIRST, et LAST, macros paramétrées, logique itérative	Une semaine ou plus
Critique	Sortie diffusée publiquement, ou alimentant un système réglementaire	Effort de recette supérieur à l’effort de réécriture

Pour un programme critique, prévoir que la validation coûtera plus cher que la réécriture. C’est normal, et c’est ce qui rend la bascule acceptable.

12.4 Accompagnement des équipes

Mettre en place un glossaire SAS vers R couvrant procédures, macros, formats et notions.

Former sur des cas réels : une procédure SAS migrée de bout en bout, avec ses contrôles.

Standardiser les sorties métier : mêmes colonnes, mêmes libellés, mêmes contrôles, pour qu’une comparaison ne demande aucun effort.

Organiser des recettes conjointes : le métier valide l’équivalence, pas seulement l’informatique.

Le point qui décide de l’adoption. Une équipe qui maîtrise SAS depuis quinze ans ne devient pas autonome en R par la seule lecture d’un guide. Prévoir un accompagnement de plusieurs semaines après la bascule, avec un référent joignable. C’est la ligne budgétaire la plus souvent oubliée et la plus déterminante.

13. Modèles et gabarits

13.1 En-tête de script

```
# -----
# Titre      : <nom du traitement>
# Objectif   : <résumé métier en une phrase>
# Entrées    : <liste + version + emplacement>
# Sorties    : <liste + format + emplacement>
# Paramètres : config/config.yml
# Règles     : <références des règles métier implémentées>
# Origine    : <programme SAS d'origine, si migration>
# Historique : <date - auteur - changement>
# -----
```

13.2 Liste de contrôle « prêt pour recette »

- Le script s'exécute de bout en bout via `Rscript`, sans intervention manuelle
- Les dépendances sont figées dans `renv.lock` et documentées
- Les entrées sont validées : présence, schéma, types
- Les contrôles qualité passent et sont journalisés : volumétrie, clés, bornes, agrégats
- Les sorties sont produites au bon format, au bon emplacement, versionnées si nécessaire
- Les pièges du chapitre 6 ont été vérifiés pour les variables concernées
- Les écarts avec SAS sont documentés dans le registre et validés par le métier
- Le programme SAS d'origine est référencé dans l'en-tête

13.3 Contrôles qualité standards

```
controles_standards <- function(df, cles, nom_etape) {
  resultats <- list(
    etape      = nom_etape,
    n_lignes   = nrow(df),
    n_colonnes = ncol(df),
    cles_nulles = sum(!stats::complete.cases(df[cles])),
    doublons   = sum(duplicated(df[cles])),
    taux_na    = vapply(df, function(x) mean(is.na(x)), numeric(1))
  )

  if (resultats$cles_nulles > 0) {
    stop(sprintf("[%s] %d cle(s) nulle(s)", nom_etape, resultats$cles_nulles),
         call. = FALSE)
  }
  if (resultats$doublons > 0) {
    warning(sprintf("[%s] %d doublon(s) sur la cle", nom_etape, resultats$doublons),
           call. = FALSE)
  }

  resultats
}
```

Annexe A. Glossaire SAS vers R

Bibliothèque (LIBNAME). En R, un dossier de fichiers ou une connexion base via `DBI`.

DATA step. Transformations ligne à ligne avec état persistant. En R, pipelines vectorisés `dplyr` ou `data.table`.

PROC. Procédure d'analyse ou de restitution. En R, packages spécialisés, et Quarto pour le rendu.

Macro. Génération de code et paramétrage. En R, fonctions, configuration et itération contrôlée.

Format et informat. Mise en forme et recodage. En R, dictionnaires et fonctions de recodage versionnés.

N. Compteur d'itérations du DATA step. Proche de `row_number()`, mais compte les passages et non les lignes.

FIRST. et LAST. Indicateurs de première et dernière observation d'un groupe trié. En R, `row_number() == 1` et `row_number() == n()` après `group_by()`.

RETAIN. Conservation d'une valeur entre observations. En R, fonctions cumulatives ou `purrr::accumulate()`.

Manquant spécial. SAS distingue `.A` à `.Z` en plus du `.` standard. R n'a que `NA` : prévoir une colonne de motif si la distinction porte du sens métier.

ODS. Système de sortie SAS. En R, Quarto ou R Markdown, avec `gt` ou `flextable`.

Annexe B. Packages de référence

Besoin	Packages
Manipulation	dplyr, tidyr, data.table
Import SAS	haven
Import texte et Excel	readr, readxl, writexl
Bases de données	DBI, dbplyr, odbc
Chaînes de caractères	stringr, stringi
Dates	lubridate
Reproductibilité	renv, here, withr
Tests	testthat
Comparaison de résultats	waldo, diffd, arsenal
Qualité de code	styler, lintr
Restitution	quarto, rmarkdown, gt, ftable, ggplot2
Journalisation	logger, futile.logger
Configuration	yaml, config, jsonlite

Annexe C. Historique des versions

Version	Date	Évolutions
1.0	27/12/2025	Version initiale : principes, structure de projet, style, équivalences, tests, performance, documentation
2.0	—	Ajout du chapitre 6 sur les pièges de sémantique ; traitement par groupe détaillé au 5.3 ; gestion des secrets au 2.4 ; outils de comparaison au 9.3 ; fiche d’écart au 9.4 ; double exécution au 9.5 ; conduite de migration au chapitre 12, avec phasage, matrice de décision et typologie de complexité ; rôles au 11.3 ; niveaux de recommandation ; correction de la numérotation des listes ; enrichissement du tableau d’équivalences et du glossaire